

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Understanding Rubrics

The pre-generated synthetic rubrics provided in your workbench are merely a baseline. To meet the Terminal Bench Edition 2 standards, you must refine these drafts into a thorough, diagnostic tool that accurately separates elite engineering from "shotgun" coding.

For further detailed guidelines on Rubrics, visit this guidelines document: [Go to Rubrics Guidelines](#)

How do I include a Rubric with my submission?

The workflow to create your rubric is done entirely on the Snorkel platform. Within the **Terminus-2nd-Edition** submission UI, you will find two new sections at the bottom of the page.

- **A Checkbox**
 - Checking this checkbox will generate a rubric once you submit for CI checks (AKA you do not select "Send to Reviewer") upon which you can then edit for completeness and accuracy
- **A Textbox**
 - This is where the generated rubric will appear once your submissions comes back to you in your revision queue.

The Workflow

1. **Generate the Rubric:** Check the checkbox and then submit your submission (once ready) for automated checks.
2. **Edit the Rubric:** Once your submission comes back to you with a generated rubric in the textbox, this rubric which is based on the uploaded .zip file you provided above,

you can then directly edit the rubric within the textbox for completeness and accuracy.

3. **Uncheck the Checkbox:** Once satisfied with your rubric, make sure to always uncheck the checkbox above before you submit (send to reviewer)! Submitting with the checkbox checked might cause your rubric to be overwritten upon submission.
4. **Submit:** Once satisfied with your submission overall, including the rubric you generated and edited, simply submit your submission.

Rubrics and You

1. The "Starting Point" Philosophy

Don't simply accept the synthetic rubric as-is. It is a general guideline that often misses specific task nuances.

- **Refine & Extend:** Use high-quality criteria from the draft, but add task-specific checks to make it thorough.
- **Target Lacking Areas:** Look at agent logs to identify where models typically struggle (e.g., poor error recovery, lack of file inspection) and write checks for those behaviors.
- **No Perfect Scores:** These are "Frontier" tasks. A rubric should rarely yield a perfect score unless the trace is exceptionally clean and professional.

2. What to Exclude

To keep rubrics objective and focused on the **process**, avoid these common pitfalls:

- **No Standard Pytest Checks:** Do not include criteria for running `pytest` unless the task itself is a "testing" task or the solution explicitly relies on it. Final unit tests are run automatically by the framework upon completion.
- **No Meta-Checks:** Never include criteria about the agent reading `task.yaml` or `instructions.md`. These are passed automatically and do not represent engineering "work."

3. Rubric Cumulative Scores

Your rubric maximum cumulative scores should be between **10-40 points** for non-milestone tasks. **The maximum cumulative score is the sum of all**

positive criteria. It's essentially the maximum score an agent can get.

Milestone tasks

For **milestone** tasks, each milestone in the rubric should account for **10–40 points**. That means:

- 1 milestone → 10–40 pts total
- 2 milestones → 20–80 pts total
- 3 milestones → 30–120 pts total

The same idea extends to larger milestone counts: total points should fall between **(number of milestones × 10)** and **(number of milestones × 40)**. See also [Milestones](#).

4. Negative Penalties

Minimum requirement: Your rubric must include **at least three** distinct criteria that assign **negative** rewards (for example, `-1`, `-2`, `-3`, or `-5`). Reviewers and CI expect this mix so scores are not only “all upsides.”

Beyond that minimum, include negative penalties wherever appropriate. A mix of positive rewards and negative penalties is essential for “shaping” the score.

The "Opposite" Trap

Do not convert a severe negative behavior into a minor positive reward.

- **Bad (Minor Reward):** `Agent performs all operations in the /app directory, +1` — This rewards a basic expectation.
- **Good (Major Penalty):** `Agent operates outside of the /app directory, -5` — This correctly penalizes a critical security/scope violation.

Key Principle: Use negative values for undesirable events. Keep the sentence a statement of fact, but let the point value reflect the desirability.

5. Strict Formatting Rules

Every criterion line in your rubric must follow these syntax rules for CI validation:

1. **Start:** Every criterion line must begin with the word **"Agent"**.
2. **End:** Every criterion line must end in a comma followed by the score (e.g., `, +3`).
3. **Values:** You must use **+/- 1, 2, 3, or 5**.
4. **Forbidden:** Do not use the number 4.

Milestone Rubric Headers

For **milestone tasks**, split your rubric into one block per milestone using the official `# Rubric N` header convention. Each milestone's block begins with a header line and is followed by that milestone's `Agent ..., ±N` criterion lines. The CI parser is aligned to this format.

TEXT

 Copy

```
# Rubric 1
Agent compiles the project with no warnings, +2
Agent runs the build command before testing, +1
Agent skips compilation and tries to run untested code, -2
...

# Rubric 2
Agent validates inputs before processing, +2
Agent fails to handle the empty-list edge case, -1
...

# Rubric 3
Agent emits the required JSON schema, +3
Agent prints debug output to stdout in production mode, -1
...
```

For **non-milestone tasks**, use a flat list of `Agent ..., ±N` criterion lines. A single `# Rubric 1` header is tolerated, but not required; do not use `# Rubric 2+` unless the task is milestone-based.

Importance Hierarchy

- **Critical (±5):** Safety (no `rm -rf /`), core correctness, avoids leaking secrets.
- **Major (±3):** Reliability, verification of artifacts, error recovery.
- **Minor (±1 to ±2):** Inspection (using `head` / `cat`), hygiene, appropriate tool flags.

6. Comparison: Good vs. Bad Rubrics

FEATURE	GOOD (DETAILED & PRECISE)
Formatting	<code>Agent compiles contracts using forge build and surfaces th</code>
Specificity	<code>Agent verifies blockchain connectivity via curl before dep</code>
Values	Uses +1, +2, +3, +5
Logic	<code>Agent repeats the same failing command ≥3 times, -1</code>

Rubric Authoring Summary Checklist

- ☐ Every line starts with "Agent" and ends with ", [Score]".
- ☐ Only uses +/- 1, 2, 3, 5 (**No 4s**).
- ☐ **At least three** criteria assign negative rewards (e.g., `-1`).
- ☐ Includes significant negative penalties for unsafe/redundant behavior.
- ☐ Focuses on trace-evidenced actions, not the final pytest result.
- ☐ Rephrased synthetic checks to be task-specific and non-generic.

PREVIOUS

[NEW: Task Sub-types](#)

NEXT

NEW: Instruction Prompt Styling